

LINUX NIVELL 2: ADMINISTRACIÓ DE SISTEMES - PREPARACIÓ PER AL

CERTIFICAT LFCS

Benvinguda i bienvenida



De Unix a Linux

- **Unix (Años 70):** Desarrollado en Bell Labs. Filosofía de utilidades pequeñas. Pasó a ser software propietario y costoso.
- **Proyecto GNU (1983):** Richard Stallman. Objetivo: sistema libre completo (Herramientas como `ls`, `grep`, GCC). Faltaba el núcleo.
- **Kernel Linux (1991):** Linus Torvalds. Núcleo libre unido a las herramientas GNU. Nacimiento de "GNU/Linux".

Sistemas Operativos Clave



- **GNU/Linux:** Sustenta el 96% de servidores web y top 500 supercomputadores. (Debian, Ubuntu, RHEL).
- **macOS:** Basado en Unix (certificado The Open Group). Comparte terminal POSIX.
- **BSD:** Unix puro libre (FreeBSD, OpenBSD). Alta seguridad y rendimiento.
- **Windows (NT):** No desciende de Unix, pero añade compatibilidad (WSL2).

Filosofía: "Todo es un archivo"

Discos (/dev/sda), procesos (/proc) y hardware se manipulan como simples archivos de texto.

Familias de Distros

Una distro añade utilidades y gestores de paquetes al kernel base.

Familia Debian

Usa `apt` y `.deb`. Estabilidad extrema. Estándar en contenedores y nube (Debian, Ubuntu).

Familia Red Hat

Usa `dnf` y `.rpm`. Enfoque corporativo/empresarial muy robusto (RHEL, Fedora).

Familia Arch

Rolling release. Control total y minimalista.

Entornos Gráficos



Servidor Gráfico

- **X11:** Estándar tradicional. Compatible pero muy pesado.
- **Wayland:** Protocolo moderno directo y seguro.

Entornos de Escritorio

- **GNOME:** Moderno, atajos de teclado (Ubuntu/Fedora).
- **KDE Plasma:** Personalizable, similar a Windows.
- **XFCE:** Muy ligero para hardware antiguo.

** Sysadmins trabajan mayormente en servidores **headless** (sin interfaz gráfica) vía SSH.*

¿Qué es un Sysadmin Moderno?

- **Puesta en Marcha:** Instalar y configurar SO Headless, cuentas y red.
- **Mantenimiento y Rendimiento:** Monitorizar RAM y discos por CLI.
- **Seguridad:** Configurar firewall, permisos, revisar logs de acceso.
- **Automatización:** Scripteado en `bash` para evitar trabajo repetitivo.

```
$ sudo apt update && apt upgrade
$ systemctl restart sshd
$ df -h /var/log
```

Certificación LFCS



Linux Foundation Certified Sysadmin (LFCS)

Práctica Real

Examen 100% basado en rendimiento en consola remota.

Tiempo

2 horas de resolución de incidencias en vivo.

Distribución

Ubuntu (Debian) o CentOS.

Clave

Memoria muscular de comandos terminal.

El Árbol de Directorios (FHS)

/ Raíz del sistema

/bin y **/sbin**: Comandos esenciales

/boot: Kernel y arranque inicial

/dev: Dispositivos de hardware

/etc: Configuración global de programas

/home: Carpetas personales de usuarios

/root: Personal del superusuario

/tmp: Archivos temporales (se borra en reinicio)

/var: Datos variables, logs, bbdd

Configuración de Red por Consola



Fundamental cuando no hay TUI o Ethernet disponible, usando

NetworkManager (nmcli) .

```
# Ver si detecta la tarjeta Wi-Fi
nmcli device status

# Escanear redes disponibles
nmcli device wifi list

# Conectar a Wi-Fi
nmcli device wifi connect "SSID" \
password "SECRETA"
```

* Opción de muy bajo nivel disponible: *wpa_supplicant*.

Nombrado de Discos

- **SATA/SSD:** `/dev/sda` , `/dev/sdb` ... (Particiones: `sda1`)
- **NVMe:** `/dev/nvme0n1` ... (Particiones: `nvme0n1p1`)

Tablas de Particiones

- **MBR:** Antiguo. Máx 4 primarias. Límite 2TB.
- **GPT:** Moderno (UEFI). 128 particiones, discos inmensos.

Filesystems

- **ext4:** Estándar de facto, journaling estable.
- **xf:** RHEL, paralelo y masivo.

Esquemas Habituales



Mínimo (VMs)

Todo en `/` (raíz). Boot EFI y Swap. Peligroso si se llena el home.

Desktop

`/home` separado en otra partición para no perder datos de usuario al formatear el SO.

Servidor (LFCS Prod)

Aísla `/var` para logs/bbdd. Evita que un log tumbe el servidor llenando el disco maestro.

- `/boot/efi` (FAT32) - 500MB
- `[SWAP]` - RAM x 1
- `/` (ext4) - 30GB
- `/var` (ext4) - 100GB

Secuencia de Arranque

1. **BIOS/UEFI**: Chequeo físico (POST) y busca gestor.
2. **GRUB2 (Bootloader)**: Configuración y sabe dónde está el Kernel.
3. **Linux Kernel**: Detecta el hardware bajo (el cerebro).
4. **Initramfs**: "Mini-Linux" en RAM para leer drivers de disco cifrado/complejo.
5. **Montar Raíz**: Disco local en "/". ***Kernel Panic*** si falla.
6. **Init (Systemd PID 1)**: Inicia servicios en paralelo.
7. **Login Prompt**: Muestra la terminal TTY.

Configuración Básica Post-Instal



■ Usuario Root vs Sudo:

Nunca entrar remoto por `root` . Usar usuario regular y elevar permisos (`sudo`).

■ Hostname:

Identificar la máquina en la red: `hostnamectl set-hostname nombre`

■ Acceso SSH (SSHD):

Herramienta absoluta de sysadmin. Acceso a terminal remota con criptografía (Man-in-the-Middle check).

```
ssh juan@192.168.1.50
```

¡Sistema Listo y Configurado!

Always has been



Wait it's all Debian?





Rutas y Jerarquía

- **Ruta absoluta:** Empieza por `/` . Ej: `/home/juan/docs/informe.txt` .
- **Ruta relativa:** Depende de dónde estás. Ej: `docs/informe.txt` , `../informe.txt` .
- **Directorio actual:** `.` **Directorio padre:** `..` .
- **Home:** `~` apunta al directorio personal del usuario.

Recordatorio LFCS

El examen asume que puedes moverte con soltura en cualquier parte del árbol de directorios sin usar exploradores gráficos.

Comandos clave de navegación



- `pwd` : Muestra dónde estás.
- `cd /ruta` , `cd ..` , `cd ~` , `cd -` .
- `ls` , `ls -l` , `ls -la` , `ls -lh` , `ls -ltr` .

Comodines (glob)

- `*.log` : Todos los archivos que terminan en `.log` .
- `file?.txt` : Coincide `file1.txt` , `fileA.txt` .
- `docs/*/index.md` : Cualquier `index.md` dentro de un subdirectorio de `docs/` .

Importante: un *glob* no es “global”. Es un patrón que el **shell expande** a una lista de archivos *antes* de ejecutar el comando.

Las rutas no son lo mismo que la variable `$PATH` : esta última es solo la lista de carpetas donde el shell busca ejecutables.

Nano: el editor amable

nano es el editor recomendado para el día a día en Debian y Ubuntu.

Muestra todos sus atajos en la parte inferior de la pantalla.

- Abrir/crear archivo: `nano /etc/ssh/sshd_config`
- **Ctrl + O**: Guardar (Write Out).
- **Ctrl + X**: Salir del editor.
- **Ctrl + W**: Buscar texto dentro del archivo.
- **Ctrl + K** / **Ctrl + U**: Cortar/pegar líneas.

Uso típico

Modificar rápidamente archivos de configuración en `/etc` (Nginx, SSH, fstab) sin necesidad de aprender modos complejos.

Vim: el omnipresente modal



Como Sysadmin, tarde o temprano caerás en un entorno donde solo existe **vi/vim**. Debes saber al menos entrar, editar unas líneas y salir con dignidad.

- **Modo Normal**: panel de mandos (inicio por defecto).
- **Modo Inserción**: escribir texto (`i` para entrar, `Esc` para salir).
- **Modo Comando**: órdenes como `:w` , `:q` , `:wq` , `:q!` .

```
# Flujo mínimo de supervivencia
$ vim /tmp/nota.txt
i # entrar en INSERT
... escribe ...
Esc Esc
:wq # guardar y salir
```

Vim Cheatsheet (mínimo LFCS)

Movimiento / edición

- `h j k l` mover
- `gg` inicio · `G` final
- `dd` borrar línea · `yy` copiar línea ·
`p` pegar
- `u` deshacer · `Ctrl+r` rehacer

Buscar / salir

- `/texto` buscar · `n/N` siguiente/
anterior
- `:w` guardar · `:q` salir
- `:wq` guardar+salir · `:q!` salir sin
guardar

Redirecciones de flujo

Todo comando nace con tres canales: `stdin` (0), `stdout` (1) y `stderr` (2). Con redirecciones decides qué se guarda, qué se muestra y qué se descarta.

- `>` Sobrescribe la salida en un archivo (creándolo si no existe).
- `>>` Añade la salida al final (modo log).
- `2>` Redirige solo los errores (`stderr`).

```
# Guardar eventos sin destruir el historial
ls -la /var/log >> inventario.log
# Buscar .conf ignorando ríos de errores
find / -name "*.conf" 2> /dev/null
```

Pipes y filtros



El operador **pipe** `|` conecta la salida de un comando con la entrada del siguiente. Es la encarnación de la filosofía Unix.

- **grep**: filtra líneas por patrón.
- **less**: paginador para salidas enormes.
- **wc -l**: cuenta líneas (muy usado tras pipes).
- **head / tail**: primeras o últimas líneas.

```
# Contar procesos nginx vivos
ps aux | grep "nginx" | wc -l
# Ver solo líneas con "Error" en un log inmenso
cat /var/log/syslog | grep "Error" | less
```



tee y xargs

- `tee` : ver en pantalla y guardar a la vez.
- `xargs` : convertir líneas (stdin) en argumentos.

```
# Guardar y ver a la vez
ps aux | grep nginx | tee /tmp/nginx.txt
# Pasar rutas a un comando
find /var/log -name "*.log" 2>/dev/null | xargs -r du -h
```

Leyendo `ls -l`

La primera columna de `ls -l` resume el tipo de archivo y los permisos para Usuario, Grupo y Otros.

- `-` archivo normal, `d` directorio, `l` enlace simbólico.
- `rwX` Usuario (propietario).
- `r-X` Grupo.
- `r--` Otros.
- Ejemplo: `-rw-r--r--` clásico de archivos de texto.

Atajo mental

Pensar siempre en tres columnas: Dueño, Grupo y Mundo. Cambiar una sin querer puede abrir un agujero de seguridad.

`chmod` y `chown` esenciales



- **Modo simbólico:** `chmod u+x script.sh` , `chmod o-r archivo` , `chmod g=r fichero` .
- **Modo octal:** `644` , `755` , `600` (sumas 4/2/1).
- **Cambiar dueño:** `sudo chown juan archivo.txt` .
- **Cambiar dueño+grupo:** `sudo chown -R www-data:www-data /var/www/html/` .

```
# Permisos típicos
chmod 755 script.sh # ejecutable para todos
chmod 644 config.txt # lectura global
chmod 600 id_rsa # clave privada SSH
```

PELIGRO: `chmod 777` regala control total a cualquiera. Nunca es la "solución mágica" en servidores de producción.



Extra LFCS

- **umask:** permisos por defecto al crear archivos/directorios (evita “chmod repetido”).
- **SGID en directorios:** heredar grupo (colaboración).
- **Sticky bit:** típico de `/tmp` , evita borrar archivos ajenos.

```
umask  
chmod g+s /ruta/directorio # SGID  
chmod 1777 /ruta/carpeta # sticky
```

Ver qué está pasando

Como Sysadmin debes poder responder en segundos: *¿qué está consumiendo la CPU y la RAM ahora mismo?*

- **Foto estática:** `ps aux` o `ps -ef`.
- Filtrar por nombre: `ps aux | grep "nginx"`.
- **Vídeo en vivo:** `top` (siempre presente), `htop` (más visual).

```
# Buscar rápidamente un proceso Python
ps aux | grep "python"
```

Matar y mandar al fondo



- **kill PID:** envía SIGTERM (15) para un cierre limpio.
- **kill -9 PID:** SIGKILL (9), destruye el proceso inmediatamente.
- **killall nombre:** mata todos los procesos con ese nombre.
- **Trabajos en segundo plano:** `comando &`, `Ctrl + Z`, `bg`, `fg`, `jobs`.

```
# Ejecutar backup sin bloquear la terminal
tar -czf backup.tar.gz /var/www/ &
# Congelar y reanudar una tarea larga
Ctrl + Z # suspende
bg # continúa en background
```




Atajos LFCS

- `pgrep -a patron` : ver PID + comando (antes de matar).
- `pkill patron` : enviar señal por patrón (con cuidado).
- `nice / renice` : bajar prioridad sin matar el proceso.

```
pgrep -a nginx  
sudo renice 10 -p 1234  
pkill -9 nginx # solo si es imprescindible
```

Archivar vs Comprimir

En Linux separarás mentalmente dos operaciones:

- **Archivar:** juntar muchos archivos en uno solo (`.tar`).
- **Comprimir:** reducir el tamaño total (`gzip` , `bzip2` , `xz`).

`tar` imprescindible

- Banderas clave: `c` (create), `x` (extract), `t` (listar), `f` (file), `v` (verbose).

```
# Crear un tar "en bruto"
tar -cvf backup.tar /etc /var/log
# Extraerlo
tar -xvf backup.tar
```

Compresión y `rsync`



- `-z` `gzip` → `.tar.gz` / `.tgz` (rápido, estándar).
- `-j` `bzip2` → `.tar.bz2` (más compresión, más lento).
- `-J` `xz` → `.tar.xz` (máxima compresión, mucha CPU).

```
# Comprimir carpeta pesada
tar -czvf datos.tar.gz /mi_carpeta/
# Extraer tar.gz
tar -xzvf datos.tar.gz
```

Copias eficientes con `rsync`

```
# Preservar permisos, dueños y fechas
rsync -av /etc/ /mnt/backup/etc/
# Hacia servidor remoto por SSH
rsync -avz /datos/ usuario@host:/backup/datos/
```

Detalle LFCS: en `rsync` , la barra final importa: `/origen/` copia *contenido*, pero `/origen` copia la carpeta como contenedor.

Inodos: el secreto bajo los nombres

Para Linux el nombre del archivo es solo una etiqueta. Los datos reales viven en un **Inodo** identificado por un número.

- Varios nombres pueden apuntar al mismo Inodo (mismos datos físicos).
- Los enlaces duros son nombres adicionales para ese mismo Inodo.

Supervivencia

Los datos se borran solo cuando eliminas el **último** enlace duro que apunta a ese Inodo.

Hard Links vs Soft Links



- **Enlace duro** (`ln`): mismo Inodo. No cruza particiones ni enlaza directorios.
- **Enlace simbólico** (`ln -s`): apunta a una ruta. Puede enlazar directorios y otros discos.
- En `ls -l`, los enlaces simbólicos empiezan por `l` y muestran `nombre → destino`.

```
# Duplicar un nombre sin duplicar datos
ln servidor.conf respaldo.conf

# Crear "acceso directo" a una carpeta web
ln -s /var/www/html/ /home/juan/acceso_web
```

Regla de oro Soft Links: crea siempre el enlace con **rutas absolutas** para evitar enlaces rotos al mover carpetas.

Manual `man` y búsqueda interna

En el examen LFCS no tendrás Google. Tu aliado absoluto es el manual integrado de Linux: **man**.

- Abrir ayuda: `man comando` (ej: `man rsync`).
- Navegación: `Espacio` (bajar), `b` (subir), flechas, `q` (salir).
- Búsqueda dentro del manual: `/palabra` , luego `n` / `N` para siguiente/anterior.

```
# Buscar "exclude" dentro de man rsync
man rsync
/exclude # saltar directo a la sección relevante
```

Encontrar comandos y docs extra



- **apropos / man -k**: buscar comandos por palabra clave.
- Ejemplo: `apropos partition` , `man -k disk` .
- **/usr/share/doc**: guías extendidas, README y ejemplos de configuración.

```
# Ver documentación específica de un paquete
ls /usr/share/doc/nginx
```

Hábito de Sysadmin: antes de buscar en la web, entrena tu reflejo de abrir `man` y `apropos` . El examen está diseñado para premiar a quien domina estas herramientas.



3.1 Anatomía del Usuario

- UID, GID, shell de login
- /etc/passwd y /etc/shadow
- useradd / usermod / userdel

3.2 Grupos y Sudo

- Grupos primarios y secundarios
- su vs sudo
- visudo y /etc/sudoers

3.3 Paquetes APT/dpkg

- apt update / install / remove
- dpkg: inspección y trazabilidad
- holds y purge

3.4 Redes: El Sistema Arterial

- Interfaces y conectividad: ip a, ip r
- Diagnóstico de puertos: ss
- ping y resolución de nombres

3.5 Criptografía Remota SSH

- Puerto 22 y autenticación
- ssh-keygen: llaves ed25519
- Hardening sshd_config

Qué es realmente un usuario

- Cada cuenta se identifica por un **UID** (User ID), no solo por el nombre.
- Está asociada a un **Shell de login** (ej: `/bin/bash` , `/usr/sbin/nologin`).
- Tiene un directorio personal **Home** propio (ej: `/home/juan`).

Archivos críticos

`/etc/passwd` (metadatos visibles) y `/etc/shadow` (hashes de contraseñas, solo root) son la columna vertebral de la autenticación.

Creación y mantenimiento de cuentas



- `useradd` : crear cuentas nuevas (UID, Home, Shell).
- `passwd` : asignar o cambiar contraseñas.
- `usermod` : modificar Shell, grupos, Home, etc.
- `userdel` : eliminar cuentas y opcionalmente su Home.

```
# Crear un usuario de trabajo
sudo useradd -m -s /bin/bash operador
sudo passwd operador
# Ver cómo lo ve el sistema
getent passwd operador
```

Grupos primarios y secundarios

- Cada usuario tiene un **grupo primario** (habitualmente del mismo nombre).
- Puede pertenecer a múltiples **grupos secundarios** para heredar permisos.
- Ver pertenencias: `id usuario` , `groups usuario` .

```
# Añadir un usuario a un grupo extra
sudo usermod -aG sudo operador
# Verificar grupos
id operador
```

su vs sudo y /etc/sudoers



- **su**: cambia de identidad completa (ej: a `root`) pidiendo su contraseña.
- **sudo**: presta privilegios temporales para un comando concreto.
- El archivo más delicado: `/etc/sudoers` , siempre editar con `visudo` .

Un error de sintaxis en `/etc/sudoers` puede dejarte sin acceso administrativo remoto en pleno examen. **visudo** valida antes de guardar.

Gestión con apt

- `apt update` : sincroniza los metadatos del repositorio.
- `apt install` / `apt remove` / `apt purge` .
- `apt-mark hold` : congela un paquete para que **ningún upgrade** lo toque.
- `apt-cache show` : inspecciona descripción, versión y dependencias **antes** de instalar.

```
sudo apt update && sudo apt install curl
sudo apt-mark hold curl
apt-mark showhold
sudo apt-mark unhold curl
```

Inspección con dpkg



- `dpkg -l` : estado de paquetes — `ii` instalado, `rc` eliminado con configs, `un` no instalado.
- `dpkg -L paquete` : ficheros que instala ese paquete.
- `dpkg -S /ruta/binario` : a qué paquete pertenece un fichero.

```
# ¿Quién instaló /usr/sbin/sshd?
dpkg -S /usr/sbin/sshd
# ¿Qué ficheros instala ese paquete?
dpkg -L openssh-server
```

remove deja configs en disco (estado `rc`). **purge** las elimina definitivamente. En producción, usa **purge** cuando retiras un servicio para siempre.

Interfaces y conectividad básica

- Interfaces típicas: `eth0` , `ens33` , `wlan0` , `lo` (loopback).
- Inspeccionar IPs y estados con `ip a` .
- Comprobar ruta por defecto (gateway) con `ip r` .

```
# Ver interfaces y rutas
ip a
ip r
# Probar conectividad
ping -c 4 8.8.8.8
```

Diagnóstico de puertos



- Ver quién escucha en qué puerto usando `ss` (reemplazo moderno de `netstat`).
- Filtrar por servicio o puerto para localizar demonios sospechosos.

```
# Mostrar puertos TCP/UDP en escucha
sudo ss -tulpn
```

Más allá de las contraseñas

- SSH escucha por defecto en el puerto **22/TCP**.
- Las contraseñas son débiles ante ataques de diccionario masivos.
- Las **llaves públicas/privadas** permiten autenticación fuerte sin exponer secretos por la red.

```
# Generar un par de llaves moderno
ssh-keygen -t ed25519 -C "lfcs@deb01"
# Copiar la llave pública al servidor remoto
ssh-copy-id juan@192.168.1.50
```

Endureciendo sshd



- Archivo de configuración: `/etc/ssh/sshd_config` .
- Buenas prácticas: deshabilitar login directo de `root` , forzar autenticación por llaves.
- Aplicar cambios con `systemctl reload sshd` o `systemctl restart sshd` .

Tras endurecer SSH, **prueba siempre en otra sesión** antes de cerrar tu conexión actual, para no bloquearte fuera del servidor.



Las 5 estaciones del arranque

1. **BIOS/UEFI**: autochequeo POST y selección de dispositivo de arranque.
2. **GRUB2**: gestor de arranque que sabe dónde está el Kernel.
3. **Kernel Linux**: detecta hardware y monta sistemas básicos.
4. **Initramfs**: mini Linux en RAM con drivers críticos.
5. **Systemd**: arranca servicios y muestra el prompt de login.

Mentalidad de rescate



- Si algo falla antes de montar `/`, obtendrás un **Kernel Panic**.
- Conocer en qué estación se rompe guía tu estrategia de recuperación.
- GRUB permite entrar en modos de rescate para manipular el resto de fases.

Parámetros de arranque

- Archivo principal: `/etc/default/grub` .
- Aplicar cambios con `update-grub` tras editar.
- Parámetros temporales se pueden inyectar directamente desde el menú de GRUB.

```
# Editar en Debian/Ubuntu
sudo nano /etc/default/grub
sudo update-grub
```

Trucos de rescate y módulos



- Recuperar acceso a root con el parámetro `init=/bin/bash` desde la línea de GRUB.
- Inspeccionar módulos del kernel cargados con `lsmod` .
- Cargar módulos manualmente con `modprobe` cuando falten drivers.

Usar `init=/bin/bash` es una técnica de rescate poderosa pero peligrosa: requiere proteger físicamente GRUB con contraseña en servidores de producción.

Daemon y ciclo de vida

- Un **daemon** es un proceso en segundo plano sin interfaz interactiva.
- `systemctl` controla estados: `start` , `stop` , `restart` , `reload` , `status` .
- **restart** apaga y enciende; **reload** relee configuración sin cortar conexiones (si el servicio lo soporta).

```
# Ciclo típico para Nginx
sudo systemctl status nginx
sudo systemctl reload nginx
sudo systemctl restart nginx
```

Runlevels modernos y arranque automático



- Hacer que un servicio sobreviva a reinicios: `enable` vs `disable` .
- Conocer `multi-user.target` (modo servidor) vs `graphical.target` (escritorio).

```
# Activar al arranque
sudo systemctl enable nginx
# Cambiar target por defecto a modo texto
sudo systemctl set-default multi-user.target
```

/var/log: cementerio tradicional

- Ubicación clásica de logs: `/var/log` .
- Ficheros clave: `syslog` , `auth.log` , `dmesg` .
- Usa `less` , `grep` , `tail -f` para inspeccionarlos.

journalctl: la caja negra



- Ver historial completo: `journalctl` .
- Filtrar por servicio: `journalctl -u nginx` .
- Filtrar por severidad: `journalctl -p err` .
- Filtrar por tiempo: `journalctl --since "yesterday"` .

```
# Ver errores recientes de un servicio
journalctl -u ssh -p err --since "1 hour ago"
```

Anatomía de /etc/fstab

- `mount` es temporal: desaparece al reiniciar.
- `/etc/fstab` registra montajes permanentes (6 columnas).
- Usar siempre **UUID** en vez de `/dev/sdX` (nombres inestables).
- Verificar antes de reiniciar con `mount -a`.

```
# /etc/fstab – formato de 6 columnas
UUID=xxxx /mnt/datos ext4 defaults,nofail 0 2
# Obtener UUID
sudo blkid /dev/sdb1
# Verificar sin reiniciar
sudo mount -a
findmnt --verify
```

Opciones clave y diagnóstico



- `defaults` : rw, exec, auto, async.
- `nofail` : arranca aunque el dispositivo no esté (USB, NFS).
- `noexec` : prohíbe ejecutar binarios (seguridad en `/tmp`).
- `ro` : solo lectura.

Un UUID incorrecto en fstab puede impedir el arranque y lanzar una shell de emergencia. Remonta raíz en rw con `mount -o remount,rw /`, corrige y reinicia.

Cambios temporales y permanentes

- El kernel expone parámetros ajustables en `/proc/sys/`.
- `sysctl -w` : cambio inmediato pero efímero (no sobrevive al reinicio).
- Persistencia: fichero `.conf` en `/etc/sysctl.d/` + `sysctl --system`.

```
# Temporal
sudo sysctl -w net.ipv4.ip_forward=1
# Permanente
sudo nano /etc/sysctl.d/99-servidor.conf
    net.ipv4.ip_forward = 1
    vm.swappiness = 10
sudo sysctl --system
```

Parámetros LFCS más relevantes



- `net.ipv4.ip_forward` : habilita enrutamiento (VPN, router).
- `vm.swappiness` : agresividad del swap (10 = servidores BD).
- `net.ipv4.tcp_syncookies` : protección SYN flood.
- `fs.file-max` : límite global de ficheros abiertos.

```
# Leer un parámetro
sysctl net.ipv4.ip_forward
# Buscar por patrón
sysctl -a | grep "ipv4"
```

Arquitectura PAM

- PAM es el portero entre cualquier servicio y el sistema de autenticación.
- Cada servicio tiene su fichero en `/etc/pam.d/` (`sshd` , `sudo` , `login`).
- Los ficheros `common-*` afectan a todo el sistema.
- 4 tipos: `auth` , `account` , `password` , `session` .

```
# Política de contraseñas - /etc/security/pwquality.conf
minlen = 12
dcredit = -1 # mínimo 1 dígito
ucredit = -1 # mínimo 1 mayúscula
maxrepeat = 3
```

Bloqueo de cuentas y límites



- `pam_faillock` : bloquea la cuenta tras N intentos fallidos.
- Config: `/etc/security/faillock.conf` (`deny=3` , `unlock_time=600`).
- `pam_limits` : límites de procesos/ficheros por usuario (`/etc/security/limits.conf`).

```
# Ver intentos fallidos de un usuario
sudo faillock --user alumno
# Desbloquear manualmente
sudo faillock --user alumno --reset
# Límites de la sesión actual
ulimit -a
```

Rutas temporales (ip route)

- El kernel consulta la tabla de enrutamiento para cada paquete saliente.
- `ip route` : cambios inmediatos pero no sobreviven al reinicio.
- Para rutas permanentes: Netplan (`/etc/netplan/*.yaml`).

```
# Ver tabla de rutas
ip route show

# Añadir ruta temporal
sudo ip route add 10.20.0.0/16 via 192.168.1.254

# Eliminar ruta
sudo ip route del 10.20.0.0/16 via 192.168.1.254

# Qué interfaz usaría para llegar a una IP
ip route get 8.8.8.8
```

Rutas permanentes y diagnóstico



- Activar IP forwarding para actuar como router:
`net.ipv4.ip_forward=1` .
- Diagnóstico por capas: IP → ruta → gateway → internet → DNS.

```
# Netplan – ruta permanente
routes:
  - to: 10.20.0.0/16
    via: 192.168.1.254

sudo netplan apply

# Diagnóstico paso a paso
ip addr show      # ¿tengo IP?
ping -c3 192.168.1.1 # ¿gateway ok?
ping -c3 8.8.8.8   # ¿internet?
ping -c3 google.com # ¿DNS?
```



Todo es un archivo en /dev

- Discos y particiones aparecen como archivos especiales en `/dev/`.
- Radar de bloques: `lsblk` para ver discos, particiones y puntos de montaje.
- Diferencias de nombres: `/dev/sda1` vs `/dev/nvme0n1p1`.

MBR vs GPT



- **MBR**: histórico, máx 4 particiones primarias y límite de ~2TB.
- **GPT**: moderno, soporta muchos más volúmenes y tamaños enormes.
- Herramientas típicas: `fdisk` para MBR, `parted` para GPT.

Cualquier escritura errónea en la tabla de particiones puede destruir instantáneamente terabytes de datos. Nunca pulses **Enter** sin revisar dos veces el disco objetivo.

Dar forma al disco crudo

- Formateadores: familia `mkfs` (`mkfs.ext4` , `mkfs.xfs` ...).
- Elegir sistema de archivos según caso de uso (ext4, xfs, etc.).

```
# Formatear una partición como ext4
sudo mkfs.ext4 /dev/sdb1
```

Montaje temporal y permanente



- Montar: `mount /dev/sdb1 /mnt/datos` .
- Desmontar con seguridad: `umount /mnt/datos` .
- Detectar UUIDs con `blkid` y definir persistencia en `/etc/fstab` .

Un error de sintaxis en `/etc/fstab` puede impedir que el sistema arranque. Úsalo con calma y valida montajes manualmente antes de fijarlos.

Virtualización del almacenamiento

LVM separa la **física** del disco (discos duros reales) de la vista **lógica** que el sistema y las aplicaciones ven, permitiendo crecer y redimensionar sin apagar.

- Combinar varios discos físicos en un solo grupo de capacidad.
- Crear volúmenes lógicos para `/var`, bases de datos, etc.

Tríada: PV → VG → LV



- **PV** (Physical Volume): `pvccreate /dev/sdb1` .
- **VG** (Volume Group): `vgcreate datosVG /dev/sdb1` .
- **LV** (Logical Volume): `lvcreate -L 50G -n webLV datosVG` .
- Redimensionar en caliente con `lvextend` + `resize2fs` (u otra herramienta según FS).

```
# Extender un volumen lógico y su sistema de archivos ext4
sudo lvextend -L +20G /dev/datosVG/webLV
sudo resize2fs /dev/datosVG/webLV
```

Crear swap como fichero

- Swap = extensión de RAM en disco. Más lento pero evita el OOM Killer.
- Fichero de swap: más flexible que partición, no requiere reparticionar.
- Hacer permanente añadiendo a `/etc/fstab`.

```
# Crear y activar fichero de swap de 2 GB
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
# Permanente en fstab
/swapfile none swap sw 0 0
```

Estado y vm.swappiness



- `vm.swappiness=10` : estándar para servidores de BD (PostgreSQL, MySQL).
- `vm.swappiness=60` : valor por defecto Debian (escritorio/general).

```
# Ver estado del swap
swapon --show
free -h
# Ajustar agresividad del swap
sudo sysctl -w vm.swappiness=10
# Permanente
echo "vm.swappiness = 10" | sudo tee \
/etc/sysctl.d/99-swap.conf
```


Niveles RAID esenciales

- **RAID 1** (espejo, 2 discos): redundancia 1 disco, espacio 50%. Para SO y BD pequeñas.
- **RAID 5** (paridad, 3+ discos): redundancia 1 disco, espacio ~67-93%. Para NAS y datos.
- **RAID no es backup**: protege contra fallo de disco, no contra borrado o ransomware.

```
# Crear RAID 1
sudo mdadm --create /dev/md0 \
  --level=1 --raid-devices=2 \
  /dev/sdb /dev/sdc
# Seguir sincronización
watch cat /proc/mdstat
```

Monitorización y recuperación



- `[UU]` = ambos discos activos; `[U_]` = uno ha fallado.
- El array sigue sirviendo datos en modo degradado durante la reconstrucción.

```
# Estado del array
cat /proc/mdstat
sudo mdadm --detail /dev/md0
# Simular fallo y sustituir disco
sudo mdadm /dev/md0 --fail /dev/sdc
sudo mdadm /dev/md0 --remove /dev/sdc
sudo mdadm /dev/md0 --add /dev/sdc
# Guardar config permanente
sudo mdadm --detail --scan | \
  sudo tee -a /etc/mdadm/mdadm.conf
```

Cifrar, abrir y montar

- LUKS cifra la partición completa (AES-256-XTS). Sin la clave, solo hay datos aleatorios.
- Hasta 8 contraseñas/claves distintas por volumen (key slots).
- `luksFormat` destruye los datos — apuntar al dispositivo correcto.

```
# Instalar y formatear
sudo apt install cryptsetup
sudo cryptsetup luksFormat /dev/sdb1
# Abrir, formatear y montar
sudo cryptsetup luksOpen /dev/sdb1 datos_cifrados
sudo mkfs.ext4 /dev/mapper/datos_cifrados
sudo mount /dev/mapper/datos_cifrados /mnt/privado
# Cerrar
sudo umount /mnt/privado
sudo cryptsetup luksClose datos_cifrados
```

Arranque automático (/etc/crypttab)



- `/etc/crypttab` : equivalente al `fstab` para volúmenes cifrados.
- Con `none` : pide contraseña en el arranque. Con fichero de clave: sin prompt.

```
# /etc/crypttab
datos_cifrados UUID=xxxx none luks
# /etc/fstab
/dev/mapper/datos_cifrados /mnt/privado ext4 defaults 0 2
# Actualizar initramfs
sudo update-initramfs -u
# Verificar cabecera LUKS
sudo cryptsetup luksDump /dev/sdb1
```

Activar e inicializar cuotas

- Sin cuotas, un usuario puede llenar el disco y derribar todos los servicios.
- Se activan por sistema de ficheros en `/etc/fstab` (`usrquota,grpquota`).
- **soft**: límite blando (superable durante el periodo de gracia).
- **hard**: límite duro absoluto (nunca puede superarse).

```
# fstab – añadir opciones de cuota
UUID=xxxx /home ext4 defaults,usrquota,grpquota 0 2
# Inicializar y activar
sudo apt install quota
sudo mount -o remount /home
sudo quotacheck -cugm /home
sudo quotaon /home
```

Asignar y consultar cuotas



- `edquota` : editar cuota de un usuario (abre editor con la tabla).
- `edquota -p alice bob` : copiar cuota de alice a bob.
- `repquota` : informe de todos los usuarios en un FS.

```
# Editar cuota de un usuario
sudo edquota -u alice
# blocks: 45200 soft: 512000 hard: 614400
# Copiar cuotas a otro usuario
sudo edquota -p alice bob carol
# Informe completo
sudo repquota /home
# Ver cuota propia
quota
```

Configurar el servidor

- NFS comparte directorios a través de la red para montarlos como discos locales.
- Servidor: paquete `nfs-kernel-server` , fichero `/etc/exports` .
- `root_squash` (por defecto): root del cliente se mapea a `nobody` (seguridad).

```
# Instalar y exportar
sudo apt install nfs-kernel-server
sudo nano /etc/exports
# /srv/compartido 192.168.1.0/24(rw, sync, no_subtree_check)
# Aplicar cambios sin reiniciar
sudo exportfs -ra
# Ver exportaciones activas
sudo exportfs -v
```

Configurar el cliente



- Cliente: paquete `nfs-common` .
- `_netdev` en fstab: espera a que la red esté activa antes de montar.
- Puerto NFS: 2049 (NFSv4 usa solo este, simplifica firewall).

```
# Ver qué exporta el servidor
showmount -e 192.168.1.10
# Montar temporalmente
sudo mount -t nfs 192.168.1.10:/srv/compartido \
/mnt/nfs/compartido
# Permanente en fstab
192.168.1.10:/srv/compartido /mnt/nfs/compartido \
nfs defaults,_netdev 0 0
```



dpkg vs APT

- **dpkg**: instala paquetes `.deb` sueltos, sin resolver dependencias.
- **apt**: gestor de alto nivel que busca, instala y actualiza desde repositorios.
- Fuentes de paquetes: `/etc/apt/sources.list` y `/etc/apt/sources.list.d/`.

```
# Flujo típico
sudo apt update
sudo apt install nginx
sudo apt upgrade
```

Gestión de dependencias



- Buscar paquetes: `apt search nombre`.
- Ver información detallada: `apt show paquete`.
- Reparar instalaciones rotas: `sudo apt --fix-broken install`.

En entornos de producción, limitarse a repositorios oficiales reduce drásticamente el riesgo de software malicioso o no mantenido.

Del código fuente al binario

A veces necesitarás software que no existe en los repositorios. La salida de emergencia es compilar manualmente desde el código fuente.

- Descargar el código (normalmente `.tar.gz` desde GitHub u otra fuente).
- Instalar herramientas de compilación y headers de desarrollo.

Ciclo GNU clásico



- `./configure` : detectar entorno y generar Makefiles.
- `make` : compilar el código.
- `sudo make install` : instalar binarios en el sistema.
- Diagnosticar librerías faltantes con `ldd binario` .

```
# Ejemplo mínimo (en carpeta del código)
./configure
make
sudo make install
```

Relojes de hardware vs sistema

- El **hardware clock** (RTC) vive en la placa base.
- El **system clock** es la hora que usa el kernel.
- `timedatectl` muestra y ajusta estados, zona horaria y NTP.

```
# Ver estado general de tiempo
timedatectl

# Cambiar zona horaria
sudo timedatectl set-timezone Europe/Madrid
```

Sincronización NTP



- Servicios típicos: `chrony` o `systemd-timesyncd`.
- Sincronizar el reloj del servidor con relojes atómicos de Internet evita errores en certificados y correlación de logs.

En clústeres y entornos con TLS, un desfase de minutos puede romper autenticaciones o hacerte diagnosticar problemas inexistentes. **Siempre** activa NTP.

Cron clásico

- Sintaxis de los 5 campos: `minuto hora día-mes mes día-semana` .
- Editar tareas del usuario actual: `crontab -e` .
- Ver tareas: `crontab -l` .

```
# Backup todos los días a las 03:00
0 3 * * * /usr/local/bin/backup.sh
```

at y systemd timers



- **at**: ejecutar una tarea única en el futuro.
- **systemd-timers**: alternativa moderna y robusta a cron, integra logs y dependencias de servicios.

En sistemas nuevos, preferir **systemd timers** para tareas críticas: se integran con `systemctl` , sobreviven mejor a reinicios y se auditan con `journalctl` .

Estructura y variables

- `#!/usr/bin/env bash` — shebang obligatorio en la primera línea.
- `set -euo pipefail` — salir ante errores, variables no definidas y fallos en pipes.
- `$1`, `$2`, `#$`, `$@` — argumentos posicionales y conteo.
- `$?` — código de salida del último comando (0 = éxito).

```
#!/usr/bin/env bash
set -euo pipefail
# Variables y captura de comando
nombre="servidor-web"
fecha=$(date +%Y-%m-%d)
# Verificar argumentos
if [[ $# -lt 2 ]]; then
    echo "Uso: $0 <usuario> <dir>"
    exit 1
fi
# Hacer ejecutable
chmod +x script.sh && ./script.sh
```

Condicionales, bucles y funciones



- Comparación numérica: `-eq -ne -lt -gt` ; cadenas: `= ≠ -z -n` ; ficheros: `-f -d -e` .
- `for servicio in nginx ssh; do ... done` — iterar servicios.
- `while IFS= read -r linea; do ... done < fichero` — leer línea a línea.
- `local var=$1` — variables locales dentro de funciones.
- `trap "rm -rf $tmp" EXIT` — limpiar al salir.

```
# Monitorización de disco
uso=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
if [[ $uso -gt 90 ]]; then
    echo "CRÍTICO: Disco al ${uso}%"; exit 2
elif [[ $uso -gt 75 ]]; then
    echo "AVISO: Disco al ${uso}%"; exit 1
fi
# Comprobar servicios en bucle
for svc in nginx ssh cron; do
    systemctl is-active --quiet "$svc" \
        && echo "$svc: OK" || echo "$svc: CAÍDO"
done
```



Mínimo privilegio

- Inicio de sesión directo como `root` está **prohibido** en entornos serios.
- Usar cuentas normales + `sudo` para elevar privilegios temporalmente.
- Cada servicio solo debe ver los datos que realmente necesita.

Blindaje físico de arranque



- Proteger GRUB con contraseña para evitar inyecciones de `init=/bin/bash` .
- Cifrado de disco y bloqueo de BIOS/UEFI en servidores con acceso físico público.

La seguridad real empieza suponiendo que el atacante ya tiene acceso a la sala del servidor y a tu red, no al revés.

Por qué DAC no basta

Los permisos clásicos (`chmod` , `chown`) son **Discretionary Access Control (DAC)**: si un demonio web es hackeado, actuará con todos los permisos que tiene el usuario del proceso.

- Necesitamos una segunda muralla independiente del dueño del proceso.

MAC y perfiles de AppArmor



- **Mandatory Access Control (MAC)**: el kernel aplica reglas obligatorias adicionales.
- AppArmor define qué archivos y capacidades puede usar cada programa.
- Inspección rápida: `aa-status` para ver perfiles cargados y modo (enforce/complain).

```
# Ver estado de AppArmor en Debian/Ubuntu
sudo aa-status
```

Demonios en escucha

Cada servicio que "escucha" en la red abre un **socket** en un puerto TCP/UDP. Saber quién escucha dónde es la base de cualquier auditoría de seguridad.

- Puertos bien conocidos (0–1023), registrados y dinámicos.

Herramientas clave



- `ss -tulpn` : muestra sockets TCP/UDP en escucha con PID/proceso.
- `lsof -i` : lista archivos de red abiertos, útil para casos específicos.

```
# Ver qué proceso ocupa el puerto 80
sudo ss -tulpn | grep ":80"
```

iptables vs UFW

`iptables` es extremadamente potente pero complejo. Debian ofrece

Uncomplicated Firewall (ufw) como capa amigable sobre él.

- Política recomendada: **denegar por defecto**, abrir solo lo necesario.

```
# Activar UFW y permitir SSH
sudo ufw default deny incoming
sudo ufw allow OpenSSH
sudo ufw enable
```

Reglas típicas y protección



- Abrir HTTP: `sudo ufw allow 80/tcp` .
- Limitar fuerza bruta en SSH: `sudo ufw limit ssh` .
- Ver estado y reglas: `sudo ufw status verbose` .

Siempre verifica que mantienes acceso SSH desde al menos una sesión antes de cerrar la terminal donde activaste reglas nuevas de UFW.

Arquitectura y reglas básicas

- Tabla `filter` : cadenas **INPUT, FORWARD, OUTPUT**.
- Política por defecto: `ACCEPT` o `DROP` — añadir reglas `ACCEPT` antes de cambiar a `DROP`.
- `-A` añade al final; `-I` inserta al inicio; `-D` borra; `-F` vacía.
- `iptables-persistent` — persistir reglas en `/etc/iptables/rules.v4` .

```
# Ver reglas con contadores
sudo iptables -L -n -v --line-numbers
# Reglas esenciales (orden importa)
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m state \
    --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -P INPUT DROP
# Persistir
sudo netfilter-persistent save
```

nftables — el sucesor moderno



- Reemplaza iptables en Debian 10+; en Debian 11+ `iptables` es un wrapper de nft.
- Una sola herramienta sustituye iptables, ip6tables, arptables y ebtables.
- Sintaxis más clara; tablas con familia `inet` cubren IPv4 e IPv6 juntos.
- Persistir: `nft list ruleset > /etc/nftables.conf` + `systemctl enable nftables` .

```
# Ver estado de nftables
sudo nft list ruleset
# Crear tabla y cadena con política drop
sudo nft add table inet firewall
sudo nft add chain inet firewall input \
    '{ type filter hook input priority 0; policy drop; }'
# Añadir reglas
sudo nft add rule inet firewall input iif lo accept
sudo nft add rule inet firewall input \
    ct state established,related accept
sudo nft add rule inet firewall input \
    tcp dport { 22, 80, 443 } accept
```


Autenticación por clave pública

- Un servidor en internet recibe ataques al puerto 22 en menos de 60 segundos.
- `ed25519` : algoritmo moderno recomendado (más seguro y rápido que RSA).
- Clave privada: en tu portátil, nunca se comparte. Clave pública: se instala en el servidor.
- **Regla de oro:** verificar acceso con clave desde una 2ª terminal ANTES de deshabilitar contraseñas.

```
# Generar par de claves (en local)
ssh-keygen -t ed25519 -C "tu@dominio.com"
# Copiar clave pública al servidor
ssh-copy-id -i ~/.ssh/id_ed25519.pub usuario@servidor
# /etc/ssh/sshd_config - directivas clave
PermitRootLogin no
PasswordAuthentication no
MaxAuthTries 3
LoginGraceTime 20
# Verificar sintaxis y aplicar
sudo sshd -t && sudo systemctl restart ssh
```

fail2ban — segunda línea de defensa



- Monitoriza logs en tiempo real y banea IPs con comportamiento de fuerza bruta.
- Nunca editar `jail.conf` ; copiar a `jail.local` y modificar ahí.
- `bantime` , `findtime` , `maxretry` — los tres parámetros principales.
- `fail2ban-client set sshd unbanip IP` — desbanear si te bloqueas tú mismo.

```
# Instalar y configurar
sudo apt install fail2ban
sudo cp /etc/fail2ban/jail.conf \
/etc/fail2ban/jail.local
# [sshd]
# enabled = true
# bantime = 3600
# maxretry = 3
sudo systemctl enable --now fail2ban
# Verificar estado e IPs baneadas
sudo fail2ban-client status sshd
# Verificar config activa de sshd
sudo sshd -T | grep -E "permitroot|password"
```

Instalación y servicio

- Instalar desde APT: `sudo apt install nginx` .
- Gestionar servicio con `systemctl : status , start , enable` .
- Logs por defecto: `/var/log/nginx/access.log` y `error.log` .

```
# Comprobar que Nginx corre
sudo systemctl status nginx
```

Puertos y bloques servidor



- Abrir `80/tcp` en UFW para HTTP.
- Configurar bloques `server {}` con `server_name` diferentes para sitios virtuales.

```
# Ver qué escucha en 80/tcp
sudo ss -tulpn | grep ":80"
```

Componentes de LEMP

- Linux: sistema operativo base.
- Engine X (Nginx): servidor web frontal.
- MariaDB/MySQL: base de datos relacional.
- PHP-FPM: intérprete de código PHP como demonio.

Integración y permisos



- Configurar Nginx para enviar peticiones PHP a `php-fpm` mediante `fastcgi_pass` .
- Ajustar dueños y permisos de los directorios web para el usuario `www-data` .

```
# Dar propiedad total del sitio web a www-data
sudo chown -R www-data:www-data /var/www/misitio/
```

Imágenes y ciclo de vida

- Contenedores comparten el kernel del host — no son VMs (usan **namespaces** y **cgroups**).
- Imagen → contenedor = clase → instancia. Las imágenes son inmutables.
- `docker run -d --name nginx -p 8080:80 nginx` — detached, nombre, puerto host:contenedor.
- `docker exec -it contenedor bash` — acceder al interior del contenedor.

```
# Imágenes
docker pull nginx:alpine
docker images

# Ciclo de vida
docker run -d --name web -p 80:80 nginx
docker ps && docker ps -a
docker logs -f web
docker stop web && docker rm web

# Volumen bind mount
docker run -d -v /var/www:/usr/share/nginx/html:ro \
  --name web nginx

# Limpieza
docker system prune
```

Docker Compose e integración con systemd



- `compose.yaml` — define servicios, redes y volúmenes en un fichero YAML.
- `docker compose up -d` / `down` — levantar y destruir el stack completo.
- Redes personalizadas: los contenedores se comunican por nombre de servicio.
- Unit de systemd con `After=docker.service` — arranque automático al boot.

```
# compose.yaml (extracto)
services:
  web:
    image: nginx:alpine
    ports: ["80:80"]
  db:
    image: mariadb:11
    environment:
      MARIADB_ROOT_PASSWORD: secret
    volumes: [datos-db:/var/lib/mysql]
volumes:
  datos-db:

# Gestión del stack
docker compose up -d
```

Desastre 1: disco lleno

- Detectar raíz sin espacio con `df -h` .
- Localizar culpables en `/var` (logs, bases de datos) con `du -sh` .
- En entornos LVM, ampliar volúmenes con `lvextend` + `resize2fs` .

Desastre 2: arranque roto y forense



- Usar GRUB e `init=/bin/bash` para recuperar acceso en caso de MBR/contraseña dañados.
- Analizar errores recientes con `journalctl -p err` y logs clásicos.

El examen LFCS premia la capacidad de **leer evidencias** en logs y comandos de estado, no los intentos aleatorios de "probar suerte" con comandos peligrosos.

Has completado el Core LFCS

- Dominas instalación, red, almacenamiento, servicios, seguridad y troubleshooting básico.
- Estás listo para afrontar exámenes prácticos de nivel fundacional.

Siguientes montañas

- Automatización con Ansible y herramientas IaC.
- Contenedores y orquestación: Docker, Kubernetes.
- Camino hacia LFCE: diseño de arquitecturas de red a gran escala.





BARCELONACTIVA.CAT

Gràcies per la vostra participació